Reinforcement Learning (DSAI 402)

Lecture 4: Bellman Equations and Policy Iteration

Prof. Mohamed Ghalwash
mghalwash@zewailcity.edu.eg

Zewail City University

Lecture 4, Monday 12 October 2026

Agenda

Objectives: you should be able to write the Bellman equations for v_π and q_π, compute the value of a policy by repeated sweeps, improve a policy from its values, and run policy iteration on a small MDP.

- Play: the class becomes the grid and every square holds a card and an arrow
- Name it: backup, policy evaluation, policy improvement, policy iteration
- Formalize it: the Bellman equations, then policy evaluation, policy improvement and policy iteration
- Apply it: the third application paper, the AI Clinician for sepsis

The Grid Is Back and You Are the Squares

- Same map as last week, but the floor is **not slippery**. Every move goes where the arrow points.
- Same points: -1 for any move, -5 more for landing on 💥, and $+10$ more for landing on ⭐. Hitting a wall keeps you in place and costs -1.
- 11 volunteers stand in the shape of the grid. Each one is a square and holds a **card** that starts at 0 and the **arrow** shown below. The ⭐ card is always 0.

	Col 1	Col 2	Col 3	Col 4
Row 1	→ 0	→ 0	→ 0	↓ 0
Row 2	🟢 → 0	💥 → 0	→ 0	⭐
Row 3	→ 0	→ 0	💥 → 0	↑ 0

Card Rounds

- In a **card round**, every square writes a new card: the points of the move its arrow makes + the card of the square it lands on.
- Use the cards **from the last round**. Everyone shows the new card at the same time.
- Keep doing card rounds until a round changes no card.

Square	Round	Arrow lands on	Points	Card there, last round	New card
🟢 (2,1)	1	💥 (2,2)	$-1-5=-6$	0	-6
(1,3)	2	(1,4)	-1	9	8

> **Tip**
> A square only talks to its neighbors. Nobody looks at the whole map.

Card Round 1

Only the squares one move away from the ⭐, along their arrows, know about it.

	Col 1	Col 2	Col 3	Col 4
Row 1	→ -1	→ -1	→ -1	↓ 9
Row 2	🟢 → -6	💥 → -1	→ 9	⭐
Row 3	→ -1	→ -6	💥 → -1	↑ 9

Card Round 2

The squares two moves away now know about the ⭐. The others still only see costs.

	Col 1	Col 2	Col 3	Col 4
Row 1	→ -2	→ -2	→ 8	↓ 9
Row 2	🟢 → -7	💥 → 8	→ 9	⭐
Row 3	→ -7	→ -7	💥 → 8	↑ 9

Card Round 3

The squares three moves away now know about the ⭐. The arrow of 🟢 leads through the 💥 so its card is $-6 + 8 = 2$.

	Col 1	Col 2	Col 3	Col 4
Row 1	→ -3	→ 7	→ 8	↓ 9
Row 2	🟢 → 2	💥 → 8	→ 9	⭐
Row 3	→ -8	→ 2	💥 → 8	↑ 9

Card Round 4

Only (1,1) and (3,1) change. They are the farthest from the ⭐ along their arrows.

	Col 1	Col 2	Col 3	Col 4
Row 1	→ 6	→ 7	→ 8	↓ 9
Row 2	🟢 → 2	💥 → 8	→ 9	⭐
Row 3	→ 1	→ 2	💥 → 8	↑ 9

Round 5 gives the same cards so we stop. Each card is now the total points the robot collects from that square if it follows the arrows.

Arrow Rounds

- When a card round changes nothing, do an **arrow round**. Every square tries all 4 moves with the current cards and switches only if another move gives a **strictly larger** sum.
- After an arrow round, go back to card rounds.

Here is 🟢 after card round 4.

Move	↑	→	↓	←
Lands on	(1,1)	💥 (2,2)	(3,1)	wall, stays
Points + card there	$-1+6$	$-6+8$	$-1+1$	$-1+2$
Sum	**5**	2	0	1

↑ gives 5, which beats the 2 of its own arrow. So 🟢 switches to ↑.

Who Switched?

Arrow round	Who switches	Card before	Card after	Why
1	🟢 (2,1): → to ↑	2	5	Going over the top avoids the 💥
2	(3,1): → to ↑	1	4	It can now follow the new path of 🟢
3	(3,2): → to ←	2	3	The long way around beats the 💥
4	nobody			We stop

- After each switch, one card round updates the cards and the next card round changes nothing.
- Each switch makes the squares behind it better, which gives the next square a reason to switch.

The Final Arrows

	Col 1	Col 2	Col 3	Col 4
Row 1	→ 6	→ 7	→ 8	↓ 9
Row 2	🟢 ↑ 5	💥 → 8	→ 9	⭐
Row 3	↑ 4	← 3	💥 → 8	↑ 9

No square wants to switch. Later today we see why these arrows are the best possible.

Debrief

- Did any square ever see the whole map?
- What did a card mean when the card rounds stopped?
- Why did a square switch its arrow?
- Did any card ever go down after a switch?
- How did we know when to stop?
- Could you do this without knowing the points and where each move lands?

Debrief

- Did any square ever see the whole map? no, each square only used its neighbors' cards
- What did a card mean when the card rounds stopped?
- Why did a square switch its arrow?
- Did any card ever go down after a switch?
- How did we know when to stop?
- Could you do this without knowing the points and where each move lands?

Debrief

- Did any square ever see the whole map? no, each square only used its neighbors' cards
- What did a card mean when the card rounds stopped? the total points from that square if the robot follows the current arrows
- Why did a square switch its arrow?
- Did any card ever go down after a switch?
- How did we know when to stop?
- Could you do this without knowing the points and where each move lands?

Debrief

- Did any square ever see the whole map? no, each square only used its neighbors' cards
- What did a card mean when the card rounds stopped? the total points from that square if the robot follows the current arrows
- Why did a square switch its arrow? taking another move once and then following the current arrows was worth more than its own arrow
- Did any card ever go down after a switch?

- How did we know when to stop?
- Could you do this without knowing the points and where each move lands?

Debrief

- Did any square ever see the whole map? no, each square only used its neighbors' cards
- What did a card mean when the card rounds stopped? the total points from that square if the robot follows the current arrows
- Why did a square switch its arrow? taking another move once and then following the current arrows was worth more than its own arrow
- Did any card ever go down after a switch? no. Switching to a better move never makes things worse
- How did we know when to stop?
- Could you do this without knowing the points and where each move lands?

Debrief

- Did any square ever see the whole map? no, each square only used its neighbors' cards
- What did a card mean when the card rounds stopped? the total points from that square if the robot follows the current arrows
- Why did a square switch its arrow? taking another move once and then following the current arrows was worth more than its own arrow
- Did any card ever go down after a switch? no. Switching to a better move never makes things worse
- How did we know when to stop? an arrow round changed no arrow
- Could you do this without knowing the points and where each move lands?

Debrief

- Did any square ever see the whole map? no, each square only used its neighbors' cards
- What did a card mean when the card rounds stopped? the total points from that square if the robot follows the current arrows
- Why did a square switch its arrow? taking another move once and then following the current arrows was worth more than its own arrow
- Did any card ever go down after a switch? no. Switching to a better move never makes things worse
- How did we know when to stop? an arrow round changed no arrow
- Could you do this without knowing the points and where each move lands? no. Every square needed the full rules of the game, i.e. the model

You Already Used All of These

Backup

Update the value of a state from the values of the states it can reach.

- Game: writing a new card from a neighbor's card

Policy evaluation

Repeat backups for a fixed policy until the values stop changing.

- Game: the card rounds

Policy improvement

In every state, switch to the action with the best one-step look ahead.

- Game: an arrow round

Policy iteration

Evaluate, improve, and repeat until the policy stops changing.

- Game: the whole thing you just did

Bellman equation

The rule behind every card round. A value is the reward now plus the discounted value of where you land.

- Game: points + the neighbor's card

Important

The game had no dice so it was easy. Today we write the same rules with dice, i.e. with an average over what the world might do.

Recap: Return and Value

The return is the discounted sum of rewards from time t on. It has a recursive form.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = R_{t+1} + \gamma G_{t+1}$$

The value of a state is the return we expect if we start there and follow π.

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right]$$

The value of an action is the return we expect if we take a now and follow π after that.

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s,\ A_t = a \right] \qquad v_\pi(s) = \sum_a \pi(a \mid s)\, q_\pi(s, a)$$

Important
A return is what happened in one run. A value is what you expect on average.

The Bellman Equation for v_π

Start from the definition and use $G_t = R_{t+1} + \gamma G_{t+1}$.

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}_\pi \left[R_{t+1} + \gamma G_{t+1} \mid S_t = s \right] \\
&= \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \, \mathbb{E}_\pi \left[G_{t+1} \mid S_{t+1} = s' \right] \right] \\
&= \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \, v_\pi(s') \right]
\end{aligned}
$$

- The second line averages over the action we pick and over what the world does next. It uses the Markov property, i.e. the return after s' does not depend on how we got to s'.
- The third line is the definition of $v_\pi(s')$.

In words, the value of a state is the average, over what we might do and what the world might do, of the reward now plus the discounted value of where we land.

Reading the Bellman Equation

Part	What it does	In the game
$\sum_a \pi(a \mid s)$	Average over our actions	One arrow, so one term
$\sum_{s',r} p(s', r \mid s, a)$	Average over what the world does	No dice, so one term
$r + \gamma\, v_\pi(s')$	Reward now + discounted next value	Points + neighbor's card, $\gamma = 1$

For 🟢 with its first arrow →, both sums have a single term, i.e.

$$v_\pi(2,1) = (-1 - 5) + v_\pi(2,2) = -6 + 8 = 2$$

That is its card after round 4.

For a fixed policy we get one linear equation for each state. With n states there are n equations and n unknowns.

The Same Equation on the Slippery Grid

Last week the floor was slippery. Take the top right square under last week's best arrows. Its arrow is ↓ and $\gamma = 1$.

- Rolls 1 to 4 (4 out of 6): the robot lands on the ⭐ and gets $-1 + 10 = 9$.
- Roll 5 (1 out of 6): the move turns to →, a wall stops the robot, and it pays 1 and stays.
- Roll 6 (1 out of 6): the move turns to ←, and the robot pays 1 and lands one square to the left, which is worth 6.6.

$$v = \tfrac{4}{6}(9) + \tfrac{1}{6}(-1 + v) + \tfrac{1}{6}(-1 + 6.6) \quad \Rightarrow \quad v \approx 8.1$$

> **Tip**
> This is the Bellman equation for one state. The policy picks one arrow and the dice give three outcomes, so the sum over the next state has three terms.

The Bellman Equation for q_π

$$q_\pi(s,a) = \sum_{s',r} p(s',r \mid s,a) \left[r + \gamma \sum_{a'} \pi(a' \mid s')\, q_\pi(s',a') \right]$$

Both equations say the same thing. The value of a choice is the reward we get now plus the discounted value of what comes next.

We can also write q_π from v_π. We will use this form for policy improvement.

$$q_\pi(s,a) = \sum_{s',r} p(s',r \mid s,a) \left[r + \gamma\, v_\pi(s') \right]$$

> **Tip**
> In the arrow rounds, each of the 4 sums a square tried was the value of one action, i.e. q for that move.

Solving for v_π by Hand

Two states A and B, with $\gamma = 0.5$ and a fixed policy.

- From A, with probability 0.5 we stay in A with reward 0, and with probability 0.5 we go to B with reward 2.
- From B we always go to A with reward 3.

$$v(A) = 0.5\,\big(0 + 0.5\,v(A)\big) + 0.5\,\big(2 + 0.5\,v(B)\big) \qquad v(B) = 3 + 0.5\,v(A)$$

Put $v(B)$ into the first equation.

$$0.75\,v(A) = 1 + 0.25\,\big(3 + 0.5\,v(A)\big) = 1.75 + 0.125\,v(A) \quad \Rightarrow \quad v(A) = \frac{1.75}{0.625} = 2.8$$

$$v(B) = 3 + 0.5 \cdot 2.8 = 4.4$$

Check by putting the numbers back: $0.5\,(0 + 1.4) + 0.5\,(2 + 2.2) = 2.8$.

Optimal Policies and Optimal Values

Policy π is at least as good as π' if $v_\pi(s) \geq v_{\pi'}(s)$ for every state.

The optimal value functions

$$v_*(s) = \max_\pi v_\pi(s) \qquad q_*(s,a) = \max_\pi q_\pi(s,a)$$

Three facts to keep

- An optimal policy always exists for a finite MDP.
- There can be many optimal policies, but they all share the same v_* and q_*.
- At least one optimal policy is deterministic.

The Bellman Optimality Equation

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \, v_*(s') \right]$$

$$q_*(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]$$

The max makes these equations nonlinear, so we cannot solve them like the two-state example.

Once we know v_* and p, the best action in s is

$$\pi_*(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \, v_*(s') \right]$$

Once we know q_*, it is $\arg\max_a q_*(s, a)$ and we do not need p.

Tip

From Equations to Updates

We now turn the Bellman equations into update rules. This is **dynamic programming** (DP).

	By hand	Today
What we have	Equations for v_π and v_*	Update rules that compute them
How we solve	Substitution, for 2 states	Sweeps, for many states
What we need	The model $p(s', r \mid s, a)$	The same model

Dynamic programming needs two things:

1. A **known model**, i.e. we know $p(s', r \mid s, a)$. Weeks 5 to 7 drop this need.
2. A state space small enough to **store one number per state**. Weeks 9 to 13 drop this need.

Policy Evaluation

Goal: given a policy π, compute v_π.

Take the Bellman equation for v_π and use it as an update rule. The old values are on the right and the new value is on the left.

$$v_{k+1}(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \, v_k(s') \right]$$

- Start from any v_0, for example all zeros. Keep terminal states at 0.
- Each sweep updates every state once.
- As k grows, v_k gets closer to v_π.

> **Important**
> This is a card round. With dice, the new card is an average over where the robot might land.

Policy Evaluation on the Two-State Example

By hand we got $v(A) = 2.8$ and $v(B) = 4.4$. Now we get them by sweeps. Both updates use the values of sweep k, as in the game.

$$v_{k+1}(A) = 0.5\,\big(0 + 0.5\,v_k(A)\big) + 0.5\,\big(2 + 0.5\,v_k(B)\big) \qquad v_{k+1}(B) = 3 + 0.5\,v_k(A)$$

Sweep k	0	1	2	3	4	5	10
$v_k(A)$	0	1.00	2.00	2.38	2.59	2.70	2.80
$v_k(B)$	0	3.00	3.50	4.00	4.19	4.30	4.40

No algebra and no matrix inverse. We just repeat a simple update.

Iterative Policy Evaluation

```
Input: policy π, small threshold θ > 0
Set V(s) = 0 for every state s, including the terminal states

repeat
    Δ = 0
    for each non-terminal state s:
        old = V(s)
        V(s) = Σ_a π(a|s) Σ_{s',r} p(s',r|s,a) [ r + γ V(s') ]
        Δ = max(Δ, |old − V(s)|)
until Δ < θ

return V   (close to v_π)
```

- This code works **in place**. It writes the new value over the old one right away, so later states in the same sweep already see it. This uses less memory and is often faster.
- With **two arrays**, we keep the old values in one array and write the new values in another. This is what we did in the game.

Why Do the Sweeps Converge?

One sweep shrinks the largest error over all states by at least a factor γ.

$$\max_s \left|v_{k+1}(s) - v_\pi(s)\right| \;\leq\; \gamma \max_s \left|v_k(s) - v_\pi(s)\right|$$

- With $\gamma = 0.9$, each sweep keeps at most 90% of the error. After 50 sweeps less than 1% is left.
- With $\gamma = 1$ this bound says nothing. The sweeps still converge if every episode ends for sure under π, as with the arrows of our game.
- The starting values do not matter. The sweeps forget them.

> **Tip**
> In the two-state example the discount is 0.5. The largest error over A and B goes 4.4, 1.8, 0.9, 0.43, 0.21, so each sweep at least halves it.

Policy Improvement

We know v_π. In state s, what if we take action a once and then go back to π?

$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\,[r + \gamma\, v_\pi(s')]$$

If some action has $q_\pi(s,a) > v_\pi(s)$, switching to it in s gives a better policy.

So do it in every state at once. This is the **greedy policy**. Here $\pi'(s)$ is the one action that a deterministic policy picks in s.

$$\pi'(s) = \arg\max_a\ q_\pi(s,a)$$

Policy improvement theorem: the greedy policy is never worse. $v_{\pi'}(s) \geq v_\pi(s)$ for every state s.

> **Important**
> If no action beats the current one in any state, the values already solve the Bellman optimality equation, so the policy is optimal.

Policy Iteration

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \cdots \xrightarrow{I} \pi_* \xrightarrow{E} v_*$$

```
Start with any policy π
repeat
    V  = policy_evaluation(π)              # E step: card rounds
    π' = greedy policy with respect to V   # I step: arrow round
    if π' == π: stop
    π = π'
return π, V
```

- Each new policy is better in at least one state or nothing changes and we stop.
- There are finitely many deterministic policies, so policy iteration stops after a finite number of steps, usually very few.
- Start each evaluation from the last V, not from zeros. In the game, one card round was enough after each switch.

Policy Iteration on Last Week's Grid

Slippery floor ($\frac{4}{6}$ the arrow, $\frac{1}{6}$ each side) and $\gamma = 1$. Start with → **in every square** and evaluate it.

π_0: all →

	Col 1	Col 2	Col 3	Col 4
Row 1	→ -1.4	→ 0.8	→ 3.1	→ 4.0
Row 2	🟢 → -3.9	💥 → 2.3	→ 5.7	⭐
Row 3	→ -5.2	→ -4.0	💥 → 2.1	→ 4.0

Improve, Then Evaluate Again

π_1: 6 arrows change and every value goes up.

	Col 1	Col 2	Col 3	Col 4
Row 1	→ 0.4	→ 2.3	↓ 4.9	↓ 7.8
Row 2	🟢 ↑ -1.2	💥 → 3.2	→ 6.3	⭐
Row 3	↑ -2.7	→ -2.5	💥 ↑ 3.9	↑ 6.6

One More Step and We Stop

π_2: 2 arrows change. One more improvement step changes nothing so we stop.

	Col 1	Col 2	Col 3	Col 4
Row 1	→ 1.9	→ 3.9	→ 6.6	↓ 8.1
Row 2	🟢 ↑ 0.1	💥 → 3.8	→ 6.7	⭐
Row 3	↑ -1.5	→ -1.8	💥 → 4.5	↑ 6.7

This is exactly the table of best arrows from last week.

Paper: The AI Clinician for Sepsis (Nature Medicine 2018)

Komorowski et al. learned how to treat sepsis in intensive care from past hospital records.

Piece	In the paper
Data	About 17,000 ICU patients from MIMIC-III, then tested on about 79,000 from eRI
State	48 measurements per 4-hour step, grouped by k-means into 750 clusters
Action	IV fluid dose × vasopressor dose, 5 levels each, so 25 actions
Reward	$+100$ if the patient is alive at 90 days, -100 if not, only at the end
Model and solver	Transitions counted from the records, then policy iteration with $\gamma = 0.99$

Nobody gave them the model p. They estimated it from data, then ran policy iteration on it.

Questions for the Sepsis Paper

1. Is a cluster of patients a Markov state? What could be missing?
2. Why is the reward given only at the end? What would go wrong with a reward for each normal lab value?
3. The model is counted from records. What happens to a (state, action) pair that doctors almost never chose?
4. The policy cannot be tried on real patients. How did the authors evaluate it, and how much would you trust that?
5. Which steps of policy iteration would change if there were 750,000 states instead of 750?
6. What would your project mentor, as a domain expert, need to check before trusting a policy like this?

> **Tip**
> Read the follow-up critique by Jeter et al. (2019), "Does the Artificial Intelligence Clinician learn optimal treatment strategies for sepsis in intensive care?" It shows how a policy can look good on paper for the wrong reasons.

Common Mistakes

- Mixing up a return and a value. One run is not an average.
- Forgetting the sum over s'. With dice, the next state is not fixed.
- Updating terminal states. Their value stays 0.
- Mixing old and new values by accident when you meant to use two arrays.
- Forgetting the γ or the reward inside the bracket.
- Breaking ties in a different way each time so policy iteration never stops.
- Using $\gamma = 1$ to evaluate a policy that can loop forever. Its values never settle.

Practice Check

1. In the two-state example, what is $v(A)$ if $\gamma = 0$?
2. Start the two-state example from $v(A) = 2$ and $v(B) = 2$ and do one sweep of policy evaluation by hand.
3. In the game, why did (3,1) switch its arrow only after 🟢 switched?
4. Why is policy iteration sure to stop after a finite number of steps?
5. Why is q_* more useful than v_* when we do not know the dynamics?
6. In the sepsis paper, which part of the DP setup was not given and had to be estimated?

Bellman equation

A value written with the values of the next states

Bellman optimality

The same, with a max over actions

Optimal policy

The policy with the largest value in every state

Dynamic programming

Compute values from a known model with the Bellman equations

Policy evaluation

Repeat the Bellman update for v_π until it settles

Policy improvement

Act greedily with respect to v_π. It is never worse.

Policy iteration

Evaluate, improve, repeat until the policy stops changing

Model

$p(s', r \mid s, a)$, which DP needs and real problems rarely give

Before We Meet Again

1. **Read Sutton and Barto, Sections 3.5 to 3.6 and 4.1 to 4.3**.
2. **Write a short response to the sepsis paper**, with a focus on the questions on the slide.
3. **Read the Go paper for week 5**: Silver et al., Nature 529, 484-489 (2016).
4. **Project teams**: the proposal and the mentor agreement are due this week. The MDP spec, the Gymnasium environment, and a non-RL baseline are due in week 7.

Next lecture: value iteration and then learning values from played episodes without a model.

> **Tip**
> Ask yourself now if your project environment can give you the full model of its dynamics. If it can, DP is a strong baseline.

Learn More

Course Homepage